LABORATORY OBSERVATION / RECORD

Course:	Full Stack Web Development	**Experiment No.:**	2
Course Code:	23CM4121	**Date:**	01-09-2026
Student Name:	R. Sandeep	**Roll No.:**	A24126552111

1. TITLE

Design a Styled Webpage Using HTML5, CSS3 and the HTML5 Canvas Element

2. AIM

To design a visually styled webpage using HTML5 and CSS3, incorporating gradients, hover effects, flexbox-based card layouts, and box shadows, and to draw basic shapes and text on the page using the HTML5 <canvas> element with JavaScript.

3. OBJECTIVE

The objectives of this experiment are:

- To understand how internal CSS (inside a <style> block) is used to style an entire HTML document.
- To apply CSS3 features such as linear gradients, text-shadow, border-radius, box-shadow, and transitions.
- To style a navigation bar with hover effects using the :hover pseudo-class.
- To build a responsive card layout using CSS flexbox (display: flex).
- To use the HTML5 <canvas> element along with JavaScript's 2D drawing context to render shapes and text.
- To combine HTML structure, CSS presentation, and JavaScript behaviour in a single webpage.

4. THEORY

CSS3 (Cascading Style Sheets Level 3) extends earlier CSS specifications with modern visual features such as gradients, shadows, rounded corners, transitions, and flexible box layouts. Styles can be written internally inside a <style> block in the document <head>, externally in a separate .css file, or inline on individual elements; this experiment uses internal CSS to keep the styling within a single HTML file.

A **linear-gradient()** is a CSS background value that smoothly blends two or more colours along a given direction, used here for the page background and the header background. The **text-shadow** property adds a drop-shadow effect behind text to improve visual depth, while **box-shadow** applies a similar shadow effect around an element's box, giving cards a raised, three-dimensional appearance.

The **:hover** pseudo-class allows a different style to be applied to an element only while the mouse pointer is over it, which is used here to change the background and text colour of navigation links and to scale up the button slightly. The **transition** property makes such style changes animate smoothly over a specified duration instead of changing instantly.

Flexbox (display: flex) is a CSS layout model that arranges child elements (the .card divs) along a row or column, distributing space between them automatically. Here, justify-content: space-around

and flex-wrap: wrap are used to evenly space the three cards and allow them to wrap onto new lines on smaller screens.

The **HTML5 <canvas>** element provides a blank rectangular drawing surface on the page. It has no visual content by itself; instead, JavaScript is used to obtain a 2D rendering context using `canvas.getContext("2d")`, after which drawing methods can be called on that context, such as:

- **fillRect(x, y, width, height):** draws a filled rectangle at the given position and size, using the current fillStyle colour.
- **beginPath(), arc(x, y, radius, startAngle, endAngle):** begins a new drawing path and defines a circular arc (a full circle when the angle spans 0 to 2π), which is then filled using fill().
- **moveTo(x, y) and lineTo(x, y):** move the drawing pointer to a starting point and draw a straight line to an ending point, rendered using stroke() with the given strokeStyle and lineWidth.
- **font, fillStyle, and fillText(text, x, y):** set the font size/family and colour, and then render text directly onto the canvas at the given coordinates.

Together, this demonstrates the separation of concerns in modern web development: HTML provides the structure, CSS3 provides the presentation, and JavaScript (via the Canvas API) provides dynamic, programmatic drawing behaviour.

5. ALGORITHM / PROCEDURE

1. Start.
2. Create a new HTML file, for example `index.html`.
3. Add the <!DOCTYPE html> declaration, the <html> root element, and inside <head> add the charset meta tag, a responsive viewport meta tag, and a <title>.
4. Inside <head>, add a <style> block and define general styling for the <body> (margin reset, font-family, and a linear-gradient background).
5. Define styles for the <header> element, including a gradient background, centered text, padding, large font size, and a text-shadow.
6. Define styles for the <nav> element and its anchor tags, including background colour, spacing, and a :hover rule that changes background/text colour with a smooth transition.
7. Define styles for the canvas section (.canvas-section) and the <canvas> element itself (border and background colour).
8. Define styles for the card container (.container) using display: flex, justify-content: space-around, and flex-wrap: wrap, and define styles for each .card (width, border, border-radius, padding, box-shadow) and its heading/paragraph text.
9. Define styles for the <button> element (background colour, padding, border-radius) and a :hover rule that darkens the background and scales the button up slightly.
10. Define styles for the <footer> element (background colour, text colour, centered text, padding).
11. In the <body>, add the <header> with the page title text.
12. Add the <nav> element containing the Home, About, Services, and Contact links.
13. Add a <section class="canvas-section"> containing a heading and a <canvas id="myCanvas"> element with a defined width and height.
14. Add a containing three blocks, each with a heading, a descriptive paragraph, and a <button>.
15. Add the <footer> element with the copyright text.

16 Add a <script> block before the closing </body> tag.

17 Inside the script, get a reference to the canvas element using `document.getElementById("myCanvas")` and obtain its 2D drawing context using `canvas.getContext("2d")`.

18 Use ctx.fillStyle and ctx.fillRect() to draw a filled rectangle on the canvas.

19 Use ctx.beginPath(), ctx.arc(), and ctx.fill() to draw a filled circle on the canvas.

20 Use ctx.beginPath(), ctx.moveTo(), ctx.lineTo(), and ctx.stroke() to draw a coloured line on the canvas.

21 Use ctx.font, ctx.fillStyle, and ctx.fillText() to render the student's name as text on the canvas.

22 Save the HTML file and open it in a web browser.

23 Observe the styled header, navigation bar with hover effects, the canvas drawing (rectangle, circle, line, and text), the three styled cards, and the footer.

24 Test the interactive elements: hover over the navigation links and buttons to verify the hover/transition effects, and resize the browser window to verify the flexbox card layout wraps correctly.

25 Record the observed output as a screenshot for documentation.

26 Stop.

6. PROGRAM

index.html

```
<!DOCTYPE html>
<html lang="en">

<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>My Styled Webpage</title>

    <style>

        /* GENERAL STYLING */
        body {
            margin: 0;
            font-family: Arial, sans-serif;
            background: linear-gradient(to right, #dfe9f3, #ffffff);
        }

        /* HEADER */
        header {
            background: linear-gradient(to right, #4CAF50, #2E8B57);
            color: white;
            text-align: center;
            padding: 20px;
            font-size: 35px;
            text-shadow: 2px 2px 5px black;
        }

        /* NAVIGATION BAR */
        nav {
            background-color: #333;
            padding: 15px;
            text-align: center;
        }

        nav a {
            color: white;
            text-decoration: none;
            margin: 15px;
            font-size: 18px;
            padding: 8px 15px;
            transition: 0.3s;
        }

        nav a:hover {
            background-color: white;
            color: black;
            border-radius: 5px;
        }

        /* CANVAS SECTION */
        .canvas-section {
            text-align: center;
            margin: 30px;
        }

        canvas {
            border: 2px solid black;
            background-color: white;
        }

        /* CONTENT CARDS */
        .container {
            display: flex;
            justify-content: space-around;
            margin: 30px;
            flex-wrap: wrap;
        }

        .card {
```

```
            width: 250px;
            background-color: white;
            border: 2px solid #ddd;
            border-radius: 12px;
            margin: 20px;
            padding: 20px;
            text-align: center;
            box-shadow: 5px 5px 12px gray;
        }

        .card h3 {
            font-family: Georgia, serif;
            font-size: 24px;
            color: darkblue;
        }

        .card p {
            font-size: 16px;
            font-family: Verdana, sans-serif;
        }

        /* BUTTON */
        button {
            background-color: green;
            color: white;
            padding: 10px 20px;
            border: none;
            border-radius: 8px;
            cursor: pointer;
            transition: 0.3s;
        }

        button:hover {
            background-color: darkgreen;
            transform: scale(1.05);
        }

        /* FOOTER */
        footer {
            background-color: #333;
            color: white;
            text-align: center;
            padding: 20px;
            margin-top: 30px;
        }

    </style>

</head>

<body>

    <!-- HEADER -->
    <header>
        My Styled Webpage
    </header>

    <!-- NAVIGATION BAR -->
    <nav>
        <a href="#">Home</a>
        <a href="#">About</a>
        <a href="#">Services</a>
        <a href="#">Contact</a>
    </nav>

    <!-- HTML5 CANVAS -->
    <section class="canvas-section">
        <h2>HTML5 Canvas Example</h2>
        <canvas id="myCanvas" width="500" height="300"></canvas>
    </section>

    <!-- THREE CONTENT CARDS -->
    <div class="container">
```

```
        <div class="card">
            <h3>Card 1</h3>
            <p>This card demonstrates borders, padding,
            margin, border radius and box shadow.</p>
            <button>Read More</button>
        </div>

        <div class="card">
            <h3>Card 2</h3>
            <p>CSS3 provides beautiful styling features
            like gradients and hover effects.</p>
            <button>Explore</button>
        </div>

        <div class="card">
            <h3>Card 3</h3>
            <p>HTML5 Canvas allows drawing graphics
            using JavaScript.</p>
            <button>Learn More</button>
        </div>

    </div>

    <!-- FOOTER -->
    <footer>
        &copy; 2026 My Styled Webpage | Designed using HTML5 & CSS3
    </footer>

    <!-- JAVASCRIPT FOR CANVAS -->
    <script>

        var canvas = document.getElementById("myCanvas");
        var ctx = canvas.getContext("2d");

        /* Rectangle */
        ctx.fillStyle = "skyblue";
        ctx.fillRect(30, 30, 120, 70);

        /* Circle */
        ctx.beginPath();
        ctx.arc(250, 65, 40, 0, 2 * Math.PI);
        ctx.fillStyle = "orange";
        ctx.fill();

        /* Line */
        ctx.beginPath();
        ctx.moveTo(50, 180);
        ctx.lineTo(350, 180);
        ctx.strokeStyle = "red";
        ctx.lineWidth = 3;
        ctx.stroke();

        /* Name */
        ctx.font = "24px Arial";
        ctx.fillStyle = "blue";
        ctx.fillText("R.sandeep", 130, 260);

    </script>

</body>

</html>
```

7. CODE EXPLANATION

Code	Explanation
<style> block	Contains all internal CSS3 rules that style the entire page.
linear-gradient()	Creates a smooth colour transition, used for the body and header backgrounds.
text-shadow / box-shadow	Adds shadow effects to the header text and to the card elements for visual depth.
nav a:hover	Changes the link's background and text colour when the mouse hovers over it.
display: flex	Arranges the three .card elements in a responsive row that wraps on smaller screens.
button:hover	Darkens the button and slightly scales it up (transform: scale) on hover.
<canvas id="myCanvas">	Provides a blank drawing surface for JavaScript to render graphics onto.
getContext("2d")	Retrieves the 2D drawing context used to call drawing methods on the canvas.
fillRect()	Draws a filled sky-blue rectangle on the canvas.
arc() + fill()	Draws a filled orange circle on the canvas.
moveTo()/lineTo()/stroke()	Draws a red horizontal line across the canvas.
fillText()	Renders the student's name as blue text directly on the canvas.

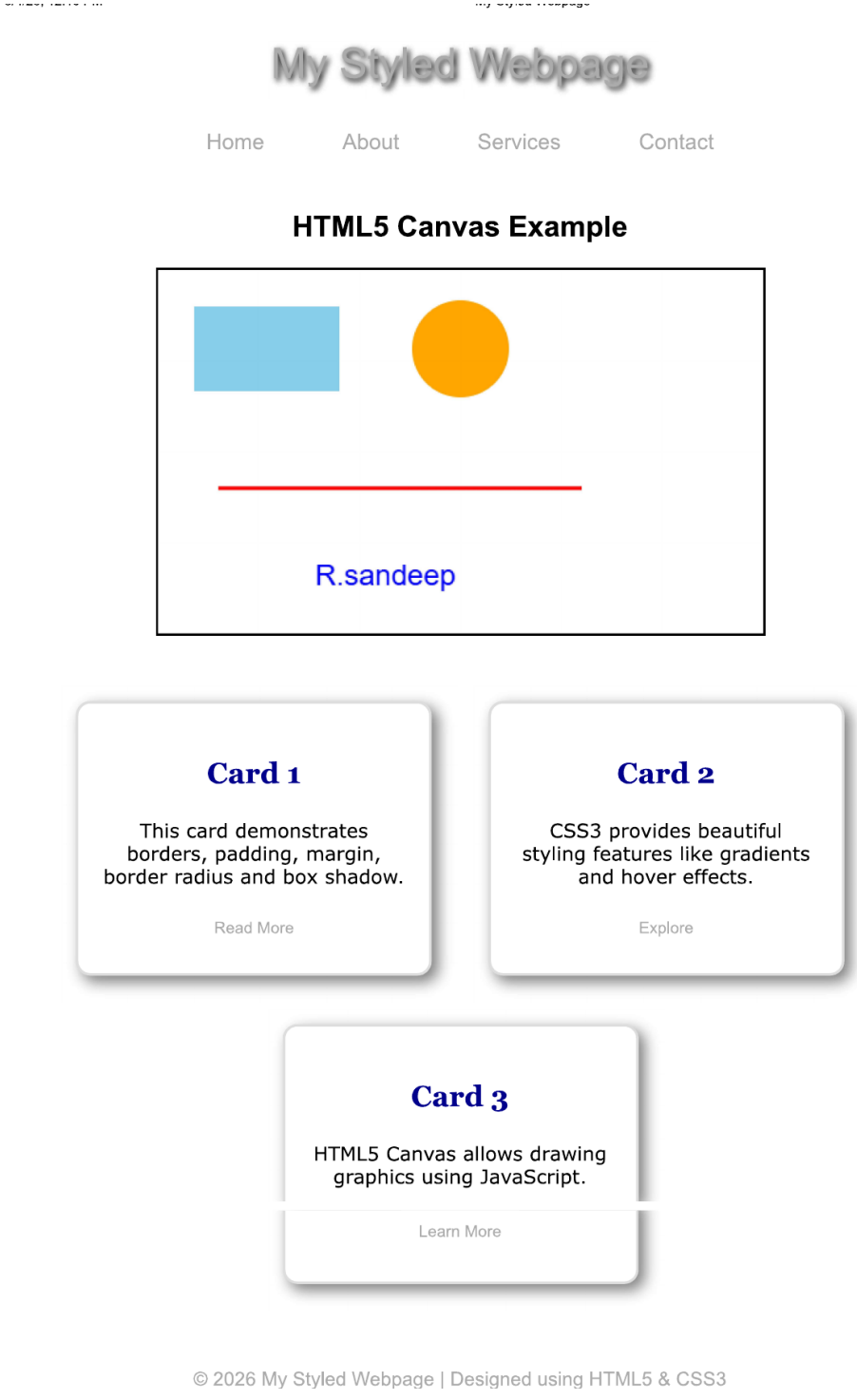
8. TEST CASES AND EXECUTION DATA

The following test cases were designed, executed by opening the page in a browser, and the actual results were recorded and compared against the expected results.

Test Case	Action	Expected Result	Actual Result	Status
TC-01	Open index.html in a browser	Page loads with gradient background, styled header, and nav bar	Page rendered correctly with styling applied	Pass
TC-02	Hover over a navigation link	Link background turns white and text turns black with smooth transition	Hover effect displayed correctly	Pass
TC-03	Observe the canvas element	Rectangle, circle, line, and name text are drawn correctly on the canvas	All canvas graphics rendered correctly	Pass
TC-04	Hover over a card button	Button darkens and scales up slightly	Hover/scale effect displayed correctly	Pass
TC-05	Resize the browser window	The three cards rearrange/wrap responsively using flexbox	Cards wrapped correctly on smaller width	Pass

9. OUTPUT

Output: Rendered Styled Webpage — The browser displayed the gradient header with the page title, the dark navigation bar with hover-highlighted links, the HTML5 canvas showing a sky-blue rectangle, an orange circle, a red line, and the name "R.sandeep" drawn in blue text, followed by three styled cards in a flexbox row, and the dark footer with the copyright text.



10. RESULT

Thus, a styled webpage was successfully designed using HTML5, CSS3, and the HTML5 Canvas element. Internal CSS3 was used to implement gradients, shadows, hover effects, transitions, and a

flexbox-based card layout, and JavaScript was used with the Canvas 2D context to draw a rectangle, a circle, a line, and custom text. The application was verified using multiple test cases, all of which produced results matching the expected output.